Precătire PA - 2022

1) Greedy:

i) Structură optimă

ii) Greedy optimal Decizia (t) optim local → optim global

↳ descompun o problemă în subprobleme

Problema rucsacului:

$\frac{v_i}{w_i}$ | $\left(\frac{v_i}{w_i}\right)$ ex: $v_1 = 100, w_1 = 100$

$v_2 = 10, w_2 = 1$

$w = 100$

$10 \cdot 100 = 1000$

w

w_0 ⟷ $(w - w_0)$

transf. apoi aceea în problema ext.

Arbori Huffman:

"ana are mere" → codificăm:

$c \rightarrow cod[c]$

cost [frază] = ?, cost min. {a:3, e:3, m:1} {m:1, r:2, -:2}

$cost[\text{frază}] = \sum_{c \in V} \text{frecă}(c) \cdot \text{lungime_cod}[c]$

(a:3) (e:3) (m:1) (m:1) (r:2) (-:2)

0 1 — 1 0

C:01

C=f[c] ...

noduri pe care le adaug la început au lungime max la final

SORTĂM CRESC DUPĂ FRECV ⇒

frecvență = de câte ori apare în text (nu raportat la nr. total de caractere)

apar mai des ↳ lungime «

0110... →

Codificarea: merg în jos până la frunză, dacă merg pe subarborele stâng, adaug 0, altfel adaug 1. În final, codificarea frunzei este secv. rezultată.

După ce am obținut codif. pentru fiecare caracter, le folosesc în fir pentru a obține decodificarea mesajului.

Decodificare: ai deja biții de la început, vezi ce secv. corespunde

Algoritm complet:

~~Construcție Huffman:~~

(a:3) (e:3) (m:1) (m:1) (r:2) (-:2)

Algoritm complet:

(a:3) (e:3) (m:1) (m:1) (r:2) (-:2)

pas 1

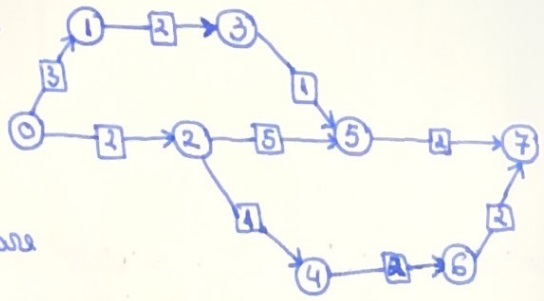
pas 2

pas 3

a:00, e:10, m:010, m:011, r:110, -:111

2) DFS / BFS
→ coadă
→ recursivitate / stivă

1.



start = 0
timp descoperire + finalizare

```
DFS: FOR v ∈ V
       If ! vis[v]
         DFS-h(v,...)

DFS-h(v,...
    found[v] = time++
    FOR neigh ∈ adj[v]
      ....
      If color[neigh]==alb
        .... (DFS-h)
    finish[v] = time++
```

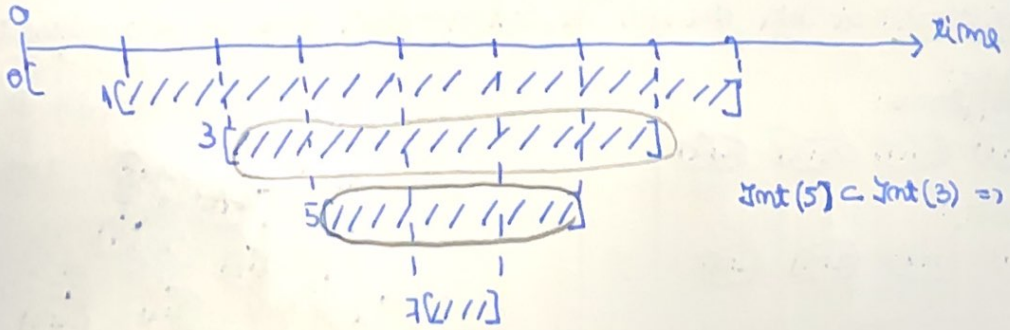
$$c[v] = \begin{cases} \text{alb}, & \text{nu am găsit nodul} \\ \text{gri}, & \text{găsit, neexplorat} \\ \text{negru}, & \text{explorat} \end{cases}$$

DFS(0): 0
DFS(1): 1 → 8
DFS(3): 2 → 7
DFS(5): 3 → 6
DFS(7): 4 → 5
DFS(2): 9 → 14
DFS(4): 10 → 13
DFS(6): 11 → 12

2. Se dă un graf... cu se clasifice muchiile:
i) arc direct (de arbore) → (nod gri, nod alb)
ii) arc invers (de ciclu) → (nod gri, nod gri)
iii) arc înainte → (nod gri, nod negru) = (u, v) ; d[u] < d[v] (l-am descop. pe u primul)
iv) arc transversal → (nod gri, nod negru) = (u, v) ; d[u] > d[v]

d[nod] = discovery time
f[nod] = finished time

TEOREMA PARANTEZELOR

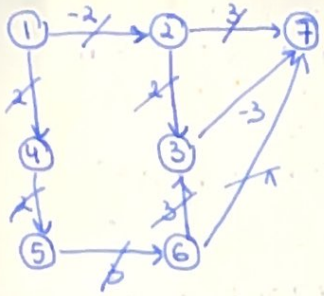


Int[5] ⊂ Int(3) ⇒ 5 este succesorul lui 3

Un nod u este succesorul lui v dacă intervalul lui u este inclus în intervalul lui v.

d[v] > d[u]
f[v] < f[u]

3.



start = 1

queue

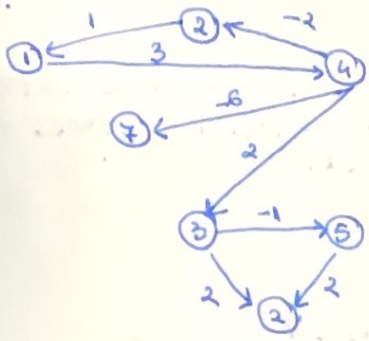


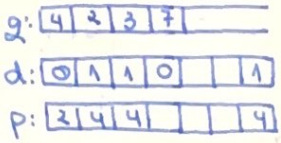
if color[u] == alb (|| visited[u] == false)

1) root 1, neigh = [2,4]
2) — 4 — 2, neigh = [3,7]

if d[node] + w[node][neigh] < d[neigh]
....

4.





1) 4 → 2, 3, 7
2) 2 → 1

? BF

5) TOPSORT → pe grafuri orientate aciclice (DAG)



{1, 2, 3}

(1, 2)
(i, j) → index i < index j

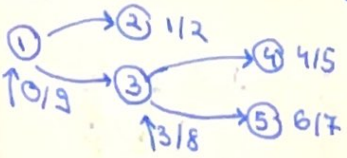


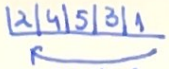
{2, 1, 3}

dc era (3 → 2) ⇒ contradicție

i) DFS
2) BFS (Kahn)

i) DFS:

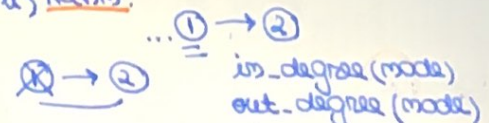




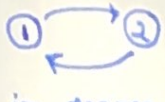
timp de finalizare → nodul curățător

```
FOR neigh ....
    .... DFS(neigh)
stk.push(node)
```

ii) Kahn:







in-degree (m) == 1

DAG → toate căile au lungime finită
cea mai lungă cale din graf: $P = (m_1, \dots m_m)$



Dacă sunt ≠ 0 ⇒ P nu mai e cea mai lungă cale ⇒

Dem:

$(\nexists)\ m_0$ a.î. in_deg(m_0) == 0

⇒ $(\exists)\ m_0$ a.î. in_degree(m_0) == 0

Pt. un graf ciclic → ce o-ar întâmpla?

DFS(1): 0 – 3 (1,2)
DFS(2): 1 – 2



1 < 2
2 < 1
⊥ ASE.

Problema nu e că nu găsește soluția, ci că soluția nu ($\exists$). Alg. întoarce ceva, dar nu e 'corect'.

Fp. că { in_degree(m_1) ≠ 0, out_degree(m_m) ≠ 0 } ⇒ $(\exists)\ m_0$ a.î. $(m_0, m_1) \in E$, $(\exists)\ m_{m+1}$ a.î. $(m_m, m_{m+1}) \in E$ ⇒

⇒ $(\exists)\ P' = (m_0, \dots, m_{m+1})$, $|P'| = |P| + 2$ ⇒ P' cea mai lungă cale ($\exists$) ⇒ { in_degree(m_1) = 0, out_degree(m_{m+1}) = 0

Complexitate: $O(|V| + |E|)$

DFS vs BFS → complex. spațială mai proastă (structuri auxiliare)
↳ consumă memorie pe stivă

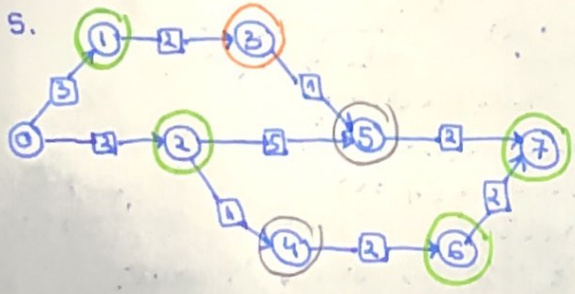
(*) Un progr. care să genereze toate sortările topologice:



{ 1,2,3
{ 1,3,2

→ backtracking în coadă

5.



m	0	1	2	3	4	5	6	7
in_degree	0	✗	✗	✗	✗	2	✗	✗
		0	0	0	0	✗	0	✗
						0		0

queue: 0 | ✗ | ✗ | ✗ | ✗ | ✗ | 6 | 7

1) scot 0
2) scot 0

0, 1, 2, 3, 4, 5, 6, 7

4) Componente conexe | tare conexe



CC: V'

$(\forall)\ x,y \in V'$, $x \to y$ (1) $\xrightarrow{DFS}$ $x \dots y$



CTC: $(\forall)\ x,y \in V'$

$\{ x \to y$
$\{ y \to x$

```
DFS:
FOR m ∈ V
    IF !vis(m)
        DFS(...)
```

↳ pt. fiecare comp. conexă

Pt. comp. tare conexe: i) Kosaraju
ii) Tarjan

i) Kosaraju:

1 - 2 - 3

(G: 3 - ...) (G^T: 1 - ...)



$G \to G^T$

(u,v) (v,u)

$3 \rightsquigarrow 1$ $1 \rightsquigarrow 3$

descrescătoare a timpilor de finalitate

1) DFS(G)
2) f[m] => modurile după f[m] descrescător (topsort)
3) DFS(G^T) după ord. sortată de la 2

Complexitate:
1) $O(|E|+|V|)$
2) $O(|E|)$
3) $O(|E|+|V|)$

~~DFS~~ DFS invers pe gf. transpus?

DFS inv pe gf. normal



DFS (0)
DFS (1): 1 - 4
DFS(2): 2 - 3
DFS(3): 5 - 4

G^T:



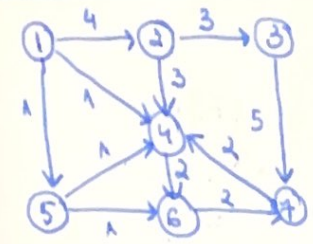
ord: 1, 2, 3
DFS(1): 1 - 4 → vis[2] = true
DFS(2): 2 - 3
DFS(3): 5 - 6

```
DFS( G=(V,E),... ) ← FOR m ∈ V
    FOR m ∈ V          vis[m] = false
        IF !vis(m)
            DFS_helpers(m,...)        O(|V|+|E|)
```

```
DFS_helpers(m, G, stack)
    .....
    FOR neigh...
    ..... ← stack.push.
```

Pași:
1) V' = TOPSORT(G)
2) $G^T = (V', E^T)$
3) DFS(G^T) ~~DFS(G)~~

i)



start = 1

pas 1) DFS(G)

DFS(1): 1-14
DFS(2): 2-11
DFS(3): 3-10
DFS(7): 4-9
DFS(4): 5-8
DFS(6): 6-7
DFS(5): 12-13

pas 2)
$V' = \{1, 5, 2, 3, 7, 4, 6\}$

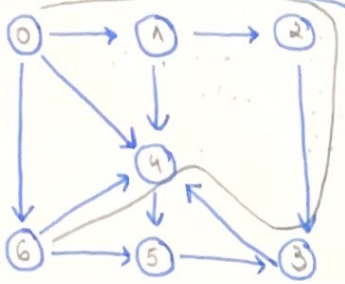
ii) Tarjan (pe exemplul de sus)

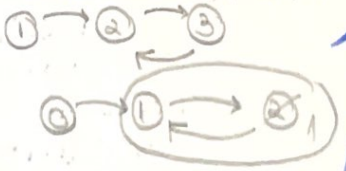
vector de low_link (din o componentă conexă)



low_link[node] = min{found[x] | x predecesor nod || x succesor nod || x == node}

found[node] == low_link[node]
low_link[node] = found[node]











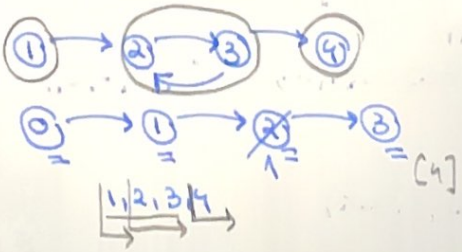


f[3] = 2
low_link[3] = 0

Tarjan(stack, ...)
FOR neigh ∈ adj[node]
IF found[node] ≠ -1
IF node ∈ stack
low_link[node] = min(low[node], low[neigh])
low_link
p[neigh] = node
Tarjan(neigh, ...)
low_link[node] = min(low_link[node], low_link[neigh])

(*) <=>
IF found[node] == low_link[node]
//stack & CTC

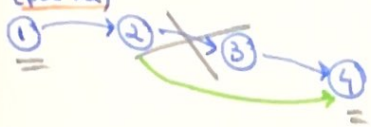
Tarjan(..., CTC)
found[...] =
low_link[...] =
CTC.push(node)



[4]

1,2,3,4

5) Muchie critică → $(\exists)\ x,y \in V$ a.î. $(\forall)$ drum de la x la y trece prin $(u,v) \Rightarrow$
(punte) $\Rightarrow (u,v)$ muchie critică



(2,3) e muchie critică dacă nu am (2,4)

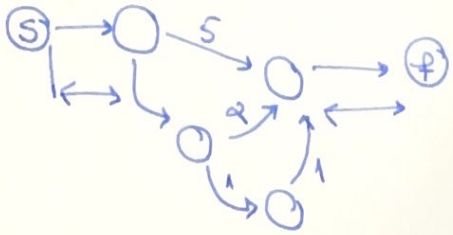
Punct de articulație → $(\exists)\ x,y \in V, x \neq y, y \neq u$ a.î. $(\forall)$ drum de la x la y trece prin u (u punct de articulație)

2 puncte de articulație

ex: pb. cu Harap Alb;



(u,v) muchie critică



Pt muchia critică:

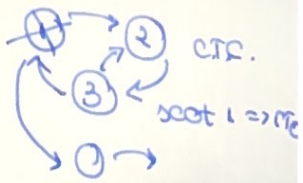
low-link[node] = min{found[x] | ...}
found[node]



low-link[neigh] > found[node]

Pt. punct de articulație:

$(\exists)$ neigh $\in$ adj[node]
1) low-link[neigh] $\geq$ found[node], node $\neq$ root, node = pct. de art.
2) node == root, children > 1



Pt. comp. biconexă (nu are niciun punct de articulație):

~~$(\exists)$~~ pct. de art. în mulțime
- dc. scot un punct, mulțimea rămâne tot. că rămâne conectată

↳ parcurg până a găsesc un punct de articulație
(p[node], node)